Master's Thesis Extension

SoC Design Ideas

Step-by-Step Implementation Guide for Beginners

How to extend the HAR + FQT + RigL on-device training project
into System-on-Chip design, FPGA prototyping, and hardware accelerator research

This guide presents six concrete ideas for expanding your HAR (Human Activity Recognition) on-device training project — which currently runs FQT (Fixed-point Quantization Training) and RigL sparse training on an STM32 Nucleo-F746ZG microcontroller — into the territory of SoC (System-on-Chip) design. Every idea is written for a reader who has never done hardware design before. Each section starts with the concept, explains why it matters for your thesis, lists the tools you will install, and then gives numbered implementation steps.

The six ideas are ordered from easiest (closest to your existing C code) to most advanced (custom silicon design). You do not need to do all six — even one idea executed well is a strong thesis extension. Start with Idea 1 or 3 if you want quick results. Start with Idea 2 or 5 if your supervisor has a hardware background.

#	Idea	Difficulty	Time estimate
1	FPGA Prototype — Deploy inference on an FPGA board	Beginner	3–4 weeks
2	Custom ISA Extension on RISC-V softcore	Intermediate	4–6 weeks
3	Hardware Resource Study — Sparsity vs LUT count	Beginner	1–2 weeks
4	Memory / DMA Architecture for On-Device Training	Intermediate	3–5 weeks
5	Power and Area Estimation (RTL → standard cell flow)	Advanced	4–8 weeks
6	SystemVerilog Verification of Quantized MAC Unit	Intermediate	3–4 weeks

All ideas connect to your existing FQT+RigL+STM32 work. Tools listed are all free and open-source unless noted.

Idea 1 Difficulty: **Beginner** Est. time: **3–4 weeks**

FPGA Prototype — Deploy HAR Inference on an FPGA

Thesis outcome: Show that your trained HAR model runs on reconfigurable hardware, compare latency and power vs. the STM32.

What is an FPGA?

An FPGA (Field-Programmable Gate Array) is a chip that contains thousands of small logic blocks connected by programmable wires. Unlike a microcontroller (like the STM32) where the hardware is fixed, you can 'program' an FPGA to become any digital circuit you want. You describe the circuit in a Hardware Description Language (HDL) — Verilog or VHDL — and the FPGA tools convert it to a configuration file that is loaded into the chip at power-on.

■ **KEY CONCEPT:** An FPGA is not programmed with C code. You describe WHAT the hardware looks like (wires, registers, logic gates), not HOW a processor executes instructions.

■ **BEGINNER NOTE:** Think of an FPGA like LEGO: you have a fixed set of blocks (logic cells), and you connect them in any pattern you want to build any circuit. A microcontroller is like a pre-built LEGO toy — faster to use but fixed.

Why This Matters for Your Thesis

Your STM32 Cortex-M7 executes the HAR model sequentially — one multiply-accumulate at a time. An FPGA can execute many operations in parallel because you define the hardware. The specific connection to your RigL work is: the RigL mask makes many weights zero at training time. If you bake the model into FPGA hardware, the synthesiser can remove the multipliers for zero weights entirely, reducing LUT count and power. This gives you a measurable hardware benefit from your sparsity training.

Tools You Will Install

Tool	Purpose	Where to get it
Vivado Design Suite (free WebPACK edition)	The main FPGA tool: synthesise, implement, and program Xilinx FPGAs	xilinx.com/support/download (free, ~30 GB install)
Verilog / VHDL	The hardware description language you will write	Comes with Vivado
Python + numpy	Generate the weight .coe files from your trained model	Already installed
GTKWave	View simulation waveforms — like an oscilloscope for simulation	gtkwave.sourceforge.net (free)
Recommended board: Arty A7-35T or Arty A7-100T	Low-cost Xilinx Artix-7 FPGA board (~$150)	digilent.com

Step-by-Step Implementation

1 Install Vivado WebPACK

Go to xilinx.com/support/download. Download Vivado ML Edition — select the WebPACK (free) licence option during installation. Choose 'Artix-7' support in the component selector to keep the download manageable (~10 GB). The install takes 30–60 minutes. When asked to activate a licence, choose 'Use WebPACK Licence' (no cost, no time limit for the devices we need).

2 Export your trained weights to a memory initialisation file

After training on PC, use your Serialize.c output or numpy to export each layer's weight tensor as a .coe file (Xilinx coefficient file format). Format: one hex value per line, preceded by 'memory_initialization_radix=16;' and 'memory_initialization_vector='. This file is loaded into FPGA block RAM at startup. Example Python snippet to generate it from a numpy float32 array: struct.pack('<f', w).hex() for each weight w.

3 Write a Verilog module for one linear layer

Start with just the final Linear layer (256 inputs to 6 outputs for HAR). A linear layer is: for each output neuron, compute dot(weights_row, input) + bias. In Verilog: an 8-bit input bus, an 8-bit weight bus (loaded from block RAM), a multiplier (DSP48 block on the FPGA), and an accumulator register. Pipeline the computation: one multiply-add per clock cycle, 256 cycles per output neuron. Begin with non-pipelined (easier): load all inputs into a register array, then sum them over 256 clock cycles using a counter.

4 Simulate your Verilog with a testbench

A testbench is a Verilog file that feeds known inputs to your module and checks the outputs. Write a testbench that applies one test vector (a known input and expected output from your trained C model). Simulate in Vivado Simulator (built-in). Open GTKWave to see the signal waveforms — inspect the accumulator counting up each cycle. This step has no hardware required, runs entirely on your PC.

5 Synthesise and check resource utilisation

In Vivado: File > New Project, add your Verilog files, set the target device to xc7a35ticsg324-1L (Arty A7-35T) or xc7a100tcsg324-1 (Arty A7-100T). Click 'Run Synthesis'. After it finishes, open 'Utilization Report'. Record: LUTs used, DSP48 blocks used, BRAM tiles used, estimated frequency. Now try sparse vs. dense: change weights so 50% are zero and re-synthesise. Does the LUT count drop? This is your key experiment.

6 Extend to the full inference pipeline

Add the Conv1d layers one at a time. Each Conv1d layer needs: a sliding-window address generator (a counter + adder), a weight BRAM indexed by [out_channel, in_channel, kernel_pos], and an accumulator. Connect the output of Conv1d to ReLU (a simple comparator: if negative, output 0). MaxPool1d is a comparison: track the maximum value in a window using a register and a comparator.

7 Program the FPGA and measure power

Connect the Arty A7 board via USB. In Vivado: Flow > Generate Bitstream, then Flow > Open Hardware Manager > Program Device. Use an ammeter (or the Arty's on-board power measurement if available) to measure current draw during inference. Compare to the STM32 current draw from your existing work. Report: mJ per inference on STM32 vs. FPGA.

Expected Results to Report in Thesis

You should be able to report a table comparing: LUT count for dense model vs. RigL-sparse model (30%, 50%, 70% sparsity), inference latency in clock cycles, estimated dynamic power from Vivado's Power Analysis report, and accuracy (unchanged, since you load the same trained weights).

Idea 2 Difficulty: **Intermediate** Est. time: **4–6 weeks**

Custom ISA Extension on a RISC-V Softcore

Thesis outcome: Design and simulate a custom RISC-V instruction that accelerates sparse quantized dot-products, measuring speedup over baseline.

What is RISC-V?

RISC-V (pronounced 'risk five') is an open-source instruction set architecture (ISA). Unlike ARM (used in the STM32 Cortex-M7), RISC-V has no licensing cost and, importantly, has a defined mechanism for adding custom instructions. A 'softcore' is a CPU implemented in VHDL/Verilog — it runs on an FPGA, not a dedicated chip. You can modify the softcore's Verilog source to add new instructions that do not exist in the standard ISA.

■ **KEY CONCEPT:** An ISA extension means you define a new machine-code instruction. The CPU's decode stage recognises your opcode and routes it to a custom execution unit you design. Software uses this instruction via an inline assembly call.

■ **BEGINNER NOTE:** Think of it like adding a new word to a language. The CPU currently understands 'multiply', 'add', 'load'. You are adding 'sparse_dot' — which does in one instruction what currently takes 256 multiply-add instructions plus a branch to skip the zeros.

Tools You Will Install

Tool	Purpose	Where to get it
RISC-V toolchain (riscv-gnu-toolchain)	Cross-compiler to build C code for RISC-V	github.com/riscv-collab/riscv-gnu-toolchain
PicoRV32 or CVA6 softcore	A RISC-V CPU in Verilog that you can modify	github.com/YosysHQ/picorv32 or github.com/openhwgroup/cva6
Verilator	Compile Verilog to C++ for fast simulation (no FPGA needed)	verilator.org (free, open-source)
Spike (RISC-V ISA Simulator)	Official RISC-V functional simulator — validate your instruction semantics	github.com/riscv-software-src/riscv-isa-sim
Vivado WebPACK (optional)	Synthesise onto FPGA if you want real hardware numbers	xilinx.com (free)

Step-by-Step Implementation

1 Install the RISC-V GNU toolchain

Clone github.com/riscv-collab/riscv-gnu-toolchain and follow the Linux build instructions. Configure with: ./configure --prefix=/opt/riscv --with-arch=rv32im --with-abi=ilp32. make -j$(nproc). This builds riscv32-unknown-elf-gcc. Test with: echo 'int main(){return 0;}' | riscv32-unknown-elf-gcc -x c - -o /dev/null.

2 Clone and run PicoRV32 in simulation

git clone https://github.com/YosysHQ/picorv32. cd picorv32 && make test — this compiles a test program with riscv-gcc, runs it through PicoRV32 in Verilog simulation, and prints PASS/FAIL. Read the picorv32.v file — it is one large Verilog file (~2500 lines). Find the instruction decode section (search for 'case (instr_' — around line 700). This is where you will add your custom opcode.

3 Define your custom instruction

RISC-V reserves the opcode space 0x0B (custom-0) for user extensions. Define: sparse_madd rd, rs1, rs2 — meaning: 'add the product rs1 * rs2 to rd, but only if rs2 (the weight) is non-zero'. The 32-bit encoding: [31:25]=0, [24:20]=rs2, [19:15]=rs1, [14:12]=000, [11:7]=rd, [6:0]=0001011 (custom-0 opcode). Write this down before touching any code.

4 Add the instruction to the PicoRV32 decode stage

In picorv32.v, find where standard instructions are decoded (the big case block). Add: if (mem_rdata_q[6:0] == 7'b0001011) begin instr_custom_sparse_madd = 1'b1; end. Add a new register: reg instr_custom_sparse_madd. In the execution stage: when instr_custom_sparse_madd is active, compute: if (rs2_val != 0) rd_val = rd_prev + rs1_val * rs2_val; else rd_val = rd_prev. Use the existing multiplier in PicoRV32 to implement the multiply.

5 Call the instruction from C using inline assembly

In your C inference code, replace the inner loop body with: .insn r 0x0B, 0, 0, acc_reg, input_reg, weight_reg — this is the GAS (GNU Assembler) inline assembly syntax for an R-type custom instruction. Wrap it in a C macro: #define SPARSE_MADD(acc, a, b) __asm__ (\" .insn r 0x0B, 0, 0, %0, %1, %2\" : \"+r\"(acc) : \"r\"(a), \"r\"(b)). Call this macro in your dot-product loop.

6 Measure speedup with Verilator simulation

Compile PicoRV32 to C++ with Verilator: verilator --cc picorv32.v --exe sim_main.cpp. Count the clock cycles your inference takes with the standard loop vs. the custom instruction. PicoRV32 has a cycle counter register (CSR) you can read before and after the loop. Calculate speedup = cycles_before / cycles_after. The custom instruction saves the branch + conditional move for zero-weight skipping.

7 Synthesise the modified PicoRV32 on FPGA (optional)

Take your modified picorv32.v into Vivado. Add it as a project source. Check that it still synthesises without errors. Look at the DSP48 count — does your multiply-accumulate unit add one DSP48 compared to unmodified PicoRV32? This tells you the hardware cost of the custom instruction.

Expected Results

A table showing: inference cycles for dense model (standard multiply loop), inference cycles for sparse model with your custom instruction, and speedup percentage. A diagram showing the custom instruction waveform from GTKWave. A cost table: extra LUTs and DSP48 blocks added by the custom execution unit.

Idea 3 Difficulty: **Beginner** Est. time: **1–2 weeks**

Hardware Resource Study — Sparsity Level vs. FPGA LUT Count

Thesis outcome: Quantify the direct silicon cost savings from RigL sparsity: show that higher sparsity produces fewer LUTs, fewer DSPs, and lower power on real FPGA synthesis — closing the loop between software training and hardware cost.

Why This is the Easiest but Most Impactful Idea

This idea does not require you to write a working hardware system. You only need to write a Verilog model of the HAR network where the weights are constants (not RAM — they are hardcoded). When weights are hardcoded zero, Vivado's synthesiser proves they can never contribute to the output and removes the associated multipliers automatically. You run synthesis at several sparsity levels and plot LUT count vs. sparsity. This graph is a direct hardware justification for the RigL algorithm you already use.

■ **KEY CONCEPT:** When a constant multiplier has value 0, any tool (synthesiser, compiler) can prove the output is always 0 and eliminate the multiply entirely. This is called 'constant propagation'. RigL creates these zeros at training time.

Step-by-Step Implementation

1 Export trained weights as Verilog constants

Write a Python script that reads your serialised model binary (from Serialize.c) and generates a Verilog module with the weights as localparams. Example output: localparam signed [7:0] W00 = 8'sd23; localparam signed [7:0] W01 = 8'sd0; where the 8-bit values are your INT8-quantised weights. Zero weights appear as 8'sd0 (or just 8'b0). Do this for the final Linear layer first (6x256 = 1536 weights).

2 Write the Verilog always block using these constants

For each output neuron n, write: always @(*) output_n = (W_n0 * input[0]) + (W_n1 * input[1]) + ... + (W_n255 * input[255]); Vivado will synthesise this as a combinational logic tree. Where W_nk == 0, the tool removes the multiplier. You do NOT need to know how to design hardware — this is just writing a mathematical formula in Verilog syntax.

3 Create a Vivado project and run synthesis at four sparsity levels

In Vivado: File > New Project > RTL Project. Add your generated Verilog file. Set target part to xc7a35t. Click Run Synthesis. After synthesis: open Utilization Report and note: LUTs, DSP48s, Slice registers, Estimated power (from Power Report). Now regenerate the Verilog file for 30%, 50%, 70%, and 90% sparsity. Re-run synthesis for each and record the numbers.

4 Plot LUT count vs. sparsity

Use matplotlib in Python to plot: x-axis = sparsity % (0, 30, 50, 70, 90), y-axis = LUT count and DSP48 count (two lines on same plot). Add a second y-axis for estimated power. The plot should show a clear downward trend — more sparsity = fewer resources. If the curve is not smooth, it means Vivado is grouping multiplications into DSP48 blocks and the granularity is coarser than individual weights.

5 Repeat for each Conv1d layer

Extend the study to the three Conv1d layers in the HAR model. Each Conv1d layer has outChannels * inChannels * kernelSize weights. For HAR: Conv1 = 32*6*3 = 576, Conv2 = 64*32*3 = 6144, Conv3 = 64*64*3 = 12288 weights. Generate Verilog for each. Note: for larger layers you may hit Vivado synthesis time limits — for 12288 constants the synthesis may take 20–40 minutes.

6 Compute energy-per-inference estimate

Vivado's Power Report gives quiescent and dynamic power estimates. Estimate clock frequency from the Timing Report (what frequency can the design achieve?). Compute: inference_time = clock_cycles / frequency. Then: energy = power * inference_time. Compare across sparsity levels. Also compare to your measured STM32 current draw.

What to Write in the Thesis

One section titled 'Hardware Resource Analysis'. Show: the Python export script, one synthesisable Verilog snippet, the LUT-vs-sparsity graph, the power-vs-sparsity graph, and a comparison table vs. the STM32. Conclude: 'RigL at X% sparsity reduces FPGA LUT usage by Y% and estimated dynamic power by Z% compared to a dense baseline, demonstrating that software sparsity decisions have a measurable impact on hardware cost.'

Idea 4 Difficulty: **Intermediate** Est. time: **3–5 weeks**

Memory / DMA Architecture for On-Device Training

Thesis outcome: Design and simulate a scratchpad memory controller that prefetches layer weights during computation, quantify bandwidth savings vs. the cache-miss-driven Cortex-M7 baseline.

The Problem This Solves

The STM32 Nucleo-F746ZG has 320 KB of SRAM. During training, CalculateGradsSequential stores all intermediate layer activations simultaneously (the VLA layerOutputs array). For the HAR model at batch=64, this uses nearly all available SRAM. The Cortex-M7 fetches weights through its cache — when the cache misses (which happens often because weight arrays are large and accessed non-sequentially), the CPU stalls. A DMA controller running in parallel with the CPU can prefetch the NEXT layer's weights into a scratchpad while the CPU is computing the CURRENT layer. This hides the memory latency.

■ **KEY CONCEPT:** DMA = Direct Memory Access. A DMA controller is a small hardware block that moves data from one memory location to another WITHOUT using the CPU. While the DMA copies weights into scratchpad, the CPU computes the current layer — both happen simultaneously, doubling effective throughput.

Tools You Will Install

Tool	Purpose	Where to get it
gem5	Full-system simulator — models CPU, cache, DMA, memory precisely	gem5.org (free, open-source)
SystemC / TLM-2.0	Transaction-level modelling — faster than gem5 for architecture exploration	systemc.org (free, IEEE standard)
ARM Cycle Models (optional)	Cycle-accurate Cortex-M7 model	armkeil.com/MDK (free evaluation)
Python matplotlib	Plot bandwidth and latency results	pip install matplotlib

Step-by-Step Implementation

1 Profile memory access patterns in your existing C code

Add instrumentation to your OnDeviceTraining C code: every time a tensor is accessed (read or written), log the base address, size, and timestamp. Run on PC with a small batch. Plot access patterns with matplotlib: x-axis = time (function calls), y-axis = memory address. This gives you the access trace that your DMA design needs to optimise.

2 Identify the bottleneck layer

From the access trace, find which layer has the largest working set (data that must be in fast memory at the same time). For HAR, Conv3 has the largest weight tensor (64*64*3 = 12288 values * 4 bytes = 48 KB). This is the target for DMA prefetching.

3 Model the scratchpad memory in SystemC

Install the SystemC library: download from accellera.org, build with CMake. Write a SystemC module sc_module for the scratchpad: it has two ports (CPU read port, DMA write port), a size_t parameter for scratchpad size, and an sc_event to signal 'prefetch done'. The CPU port: combinational read (zero latency). The DMA port: modelled as a timed write (N ns per word based on your SRAM spec).

4 Write the DMA controller module

The DMA controller is another sc_module with a simple state machine: IDLE → REQUEST (assert a bus request) → ACTIVE (transfer one word per cycle) → DONE. Parameters: src_addr (where weights live in main memory), dst_addr (scratchpad base), transfer_size (in bytes), trigger (sc_event from CPU). The CPU triggers the DMA at the START of layer N to prefetch layer N+1's weights. By the time layer N finishes, layer N+1's weights are already in scratchpad.

5 Connect CPU, DMA, scratchpad, and main memory in a top-level module

Write a sc_module top that instantiates: a CPU model (simply a timed thread that reads from scratchpad), a DMA, a scratchpad, and a main memory. Run the SystemC simulation: sc_start(simulation_time). Collect: total simulation time WITH DMA prefetching, total time WITHOUT DMA (CPU reads directly from main memory with added latency). Compute speedup.

6 Validate against STM32 measurements

Cross-check your simulation against real measurements from the STM32. Use the DWT cycle counter on the Cortex-M7 to measure the number of cycles for each Conv1d forward pass: DWT->CYCCNT before the call, DWT->CYCCNT after. If your simulation predicts X cycles and the hardware gives Y, the ratio X/Y is your simulation accuracy. Report this as a validation result.

7 Explore different scratchpad sizes

Run the simulation for scratchpad sizes from 8 KB to 128 KB. Plot speedup vs. scratchpad size. There will be a 'knee' point where more scratchpad stops giving benefit (because the whole layer's weights already fit). This tells you the minimum useful scratchpad size — a design point for any future custom chip.

Expected Thesis Contribution

A memory architecture analysis showing: access pattern heatmap from profiling, DMA prefetch speedup vs. baseline (expected 20–40% for the large layers), optimal scratchpad size for the HAR model, and a comparison of how much SRAM each training phase needs. This directly informs how a custom SoC for your model should be designed.

Idea 5 Difficulty: **Advanced** Est. time: **4–8 weeks**

Power and Area Estimation — RTL to Standard-Cell Flow

Thesis outcome: Synthesise the quantized MAC unit in a standard-cell library, extract area in um² and dynamic power in mW, project full-model silicon cost, and compare quantized vs. float32 implementations.

What is a Standard-Cell Flow?

FPGA synthesis (Idea 1) maps your design to the FPGA's fixed logic blocks. Standard-cell synthesis maps your design to a library of pre-designed transistor circuits (AND gates, flip-flops, multiplexers) on a specific silicon process. The open-source Skywater 130nm PDK (Process Design Kit) is a real commercial foundry process that Google made freely available. You can synthesise a circuit into Skywater 130nm cells and get real area (in um²) and power estimates that reflect what a custom chip would cost.

■ **CAUTION:** This idea requires more setup than the others and uses advanced tools. If you are short on time, do Idea 1 or 3 first and add this one as a stretch goal.

■ **KEY CONCEPT:** PDK = Process Design Kit. It is the description of a chip manufacturing process: what transistors are available, how fast they switch, how much power they use, and how to draw the metal wires. Skywater 130nm is a real PDK released by Google for free.

Tools You Will Install

Tool	Purpose	Where to get it
Yosys	Open-source RTL synthesis — converts Verilog to gate-level netlist	yosyshq.net (free, open-source)
OpenROAD	Place-and-route, timing, and power analysis after synthesis	theopenroadproject.org (free)
Skywater 130nm PDK	Standard cell library for a real silicon process	github.com/google/skywater-pdk (free, Apache 2.0)
OpenLane 2	Automated flow wrapping Yosys + OpenROAD + Skywater	github.com/efabless/openlane2 (free)
Docker	OpenLane runs inside a container for reproducibility	docker.com (free)

Step-by-Step Implementation

1 Install OpenLane 2 via Docker

Install Docker Desktop. Pull the OpenLane image: docker pull ghcr.io/efabless/openlane2. Clone the OpenLane2 repository: git clone https://github.com/efabless/openlane2. Run the example design to verify your install: cd openlane2 && python -m openlane designs/spm/config.json. This should complete and produce a GDS layout file. If this works, your environment is ready.

2 Write a Verilog module for the INT8 MAC unit

The core computation of your model is: accumulator += weight * input. For INT8: both weight and input are signed 8-bit integers. The product is 16-bit. The accumulator is 32-bit. Verilog: module mac_int8 (input signed [7:0] a, b, input clk, rst, output reg signed [31:0] acc); always @(posedge clk) if(rst) acc = 0; else acc = acc + a * b; endmodule. Also write a float32 version using an IEEE 754 FP multiplier (from opencores.org) for comparison.

3 Write an OpenLane config.json for your MAC module

Create a folder designs/mac_int8/. Inside: config.json with DESIGN_NAME, VERILOG_FILES (path to your .v), CLOCK_PERIOD (target in ns), PDK (sky130A). The clock period determines how fast you want the circuit — start with 10 ns (100 MHz). Lower periods give faster but larger circuits. OpenLane will try to meet your timing constraint and report if it can.

4 Run the OpenLane synthesis and place-and-route flow

python -m openlane designs/mac_int8/config.json. This runs: Yosys synthesis → floorplanning → placement → CTS (clock tree synthesis) → routing → DRC check (design rule check). Total runtime: 5–30 minutes depending on your PC. The output folder (runs/RUN_*) contains: synthesis_report (LUT count), area.rpt (um²), power.rpt (dynamic + static mW).

5 Read the reports and record key numbers

From the synthesis report: total cell count, critical path delay (the longest combinational path — this limits your maximum clock frequency). From the area report: total area in um². From the power report: dynamic power at your target frequency (mW), leakage power (mW). Repeat for: INT8 MAC, INT4 MAC, FP32 MAC. Compute: FP32_area / INT8_area ratio and FP32_power / INT8_power ratio.

6 Scale to full model

Estimate full-model silicon cost: total_MACs = sum over all layers of (output_size * input_size). For HAR: Conv1=32*6*3*L1 + Conv2=64*32*3*L2 + Conv3=64*64*3*L3 + Linear=6*256. Scale: total_area_estimate = MACs_per_cycle * mac_area_um2. For a single-MAC design: latency = total_MACs / clock_frequency. For a parallel design with N MAC units: latency = total_MACs / (N * clock_frequency). Plot: number of parallel MACs vs. area and latency.

7 Compare RigL sparse vs. dense

At 50% sparsity, approximately 50% of the MACs compute 0 * x = 0. If you implement a zero-skip circuit (check if weight == 0 before MAC), dynamic power drops because the multiplier is clock-gated. Implement clock gating: assign mac_en = (weight != 0); and use an enable signal on the flip-flop. Re-run OpenLane with clock gating enabled. Report the power difference.

Expected Thesis Results

A table comparing INT8 MAC vs. FP32 MAC: area (um²), max frequency (MHz), dynamic power at 100 MHz (mW). A projected full-model area and power estimate. A plot of parallel MAC units vs. area vs. latency showing the hardware design space. This is publishable as a hardware-efficiency analysis of quantized sparse models.

Idea 6 Difficulty: **Intermediate** Est. time: **3–4 weeks**

SystemVerilog Verification of the Quantized MAC Unit

Thesis outcome: Formally verify that the FPGA/ASIC MAC unit produces outputs that match the C reference model (OnDeviceTraining) to within the quantization error bound — demonstrating hardware correctness.

What is Hardware Verification?

When you design a hardware block in Verilog, you need to prove it computes the correct result before taping out silicon or programming an FPGA. Unlike software testing (where you can patch a bug with a software update), a bug in an ASIC is permanent — the chip is wrong. Hardware verification uses testbenches (stimulus generators + result checkers) and formal property checking to prove correctness. For your thesis: you already have a C golden model (OnDeviceTraining). You can use it as the reference against which you check your Verilog RTL.

■ **KEY CONCEPT:** A golden model is a reference implementation whose correctness you trust. You feed the same inputs to the golden model and to the RTL. If outputs match (within error tolerance), the RTL is verified. OnDeviceTraining's C code IS your golden model.

Tools You Will Install

Tool	Purpose	Where to get it
Icarus Verilog (iverilog)	Open-source Verilog simulator	iverilog.icarus.com (free)
GTKWave	Waveform viewer for simulation results	gtkwave.sourceforge.net (free)
cocotb	Python testbench framework — write testbenches in Python, not Verilog	cocotb.org (free, pip install cocotb)
Verilator (optional)	Faster compiled simulation	verilator.org (free)
SymbiYosys (optional)	Formal property checking — mathematical proof of correctness	github.com/YosysHQ/sby (free)

Step-by-Step Implementation

1 Set up cocotb with Icarus Verilog

pip install cocotb. Install Icarus: sudo apt install iverilog (Linux/WSL) or download Windows installer. Create a directory: mkdir tb_mac && cd tb_mac. Write a Makefile: TOPLEVEL_LANG=verilog, TOPLEVEL=mac_int8, MODULE=test_mac, include $(shell cocotb-config --makefiles)/Makefile.sim. Run: make. This invokes iverilog to compile your Verilog and runs your Python test.

2 Generate test vectors from your C golden model

Add a test-vector export to your OnDeviceTraining C code: before and after each linear layer call, write the input tensor and output tensor to a binary file using fwrite. Run a forward pass in PC mode. In Python, read these files with numpy: inputs = np.fromfile('layer0_input.bin', dtype=np.int8), expected = np.fromfile('layer0_output.bin', dtype=np.int32). These are your reference input/output pairs.

3 Write the cocotb testbench

In test_mac.py: import cocotb from cocotb.triggers import RisingEdge. The @cocotb.test() coroutine: set dut.rst.value=1, await RisingEdge(dut.clk), set dut.rst.value=0. Then for each (a, b) pair in your test vectors: set dut.a.value = int(a), dut.b.value = int(b), await RisingEdge(dut.clk). After N cycles, read dut.acc.value and compare to the expected accumulator value. assert abs(int(dut.acc.value) - expected[i]) == 0, f'Mismatch at {i}'.

4 Handle quantisation error in assertions

For INT8 weights * INT8 inputs accumulated in INT32, the RTL should match the C code exactly (bit-for-bit), because both use integer arithmetic. If you are comparing against a float32 reference instead, use a tolerance: assert abs(rtl_result - float_ref) / abs(float_ref) < 1e-3. Document the maximum observed error across all test vectors — this is your quantization error bound.

5 Run coverage analysis

cocotb's coverage: use cocotb_coverage.coverage import CoverPoint. Add coverage points for: weight values near INT8 max (+127, -128), input values of 0, accumulator overflow conditions. Coverage analysis tells you which input conditions your tests actually exercised. Aim for 100% line coverage of your Verilog module and >90% branch coverage (every if/else branch taken at least once).

6 Add formal property checking with SymbiYosys (stretch goal)

Formal verification proves properties for ALL possible inputs, not just your test vectors. Install: pip install symbiyosys. Write a property file (.sby): [engines] smtbmc. [script] read_verilog mac_int8.v. In your Verilog, add: assume (a != 0 || b != 0); assert (acc == $past(acc) + a*b); (the acc must equal its previous value plus the product). SymbiYosys will either prove the property or find a counterexample. A formal proof is much stronger than simulation testing.

7 Document as a verification plan

Write a one-page verification plan in your thesis: what properties were checked, how many test vectors, coverage percentage, maximum observed error vs. golden model. This section shows that you applied engineering rigour to your hardware design — an important distinction between a well-designed project and a prototype.

Expected Thesis Contribution

A verification report showing: test vector count, pass/fail summary, coverage percentage, maximum quantization error (should be 0 for integer arithmetic), waveform screenshot from GTKWave showing correct accumulation. Optional: a formal proof certificate from SymbiYosys confirming the MAC always satisfies its arithmetic property.

Getting Started — Which Idea Should You Choose First?

Do not try to do all six at once. Pick one idea that matches your timeline and your supervisor's expectations. Here is a decision guide:

If your situation is...	Start with...	Why
You have 1–2 weeks and want quick results	Idea 3 — Hardware Resource Study	Only needs Vivado + Python you may already have. Synthesises your C model's weights directly.
Your supervisor is a hardware engineer	Idea 1 — FPGA Prototype	Most tangible result: a running system on real hardware. Easy to demo.
You want the most academic novelty	Idea 2 — RISC-V ISA Extension	Custom ISA extensions are an active research area. Strong publishability.
You want to understand memory bottlenecks	Idea 4 — DMA Architecture	Directly motivated by the SRAM constraints you already hit on the STM32.
You want to compare INT8 vs. FP32 in silicon	Idea 5 — Power and Area Estimation	Gives the most concrete hardware cost numbers. Strongest thesis claim.
You want to prove your hardware is correct	Idea 6 — SystemVerilog Verification	Most rigorous. Pairs well with Idea 1 or 5 as the verification step.

Recommended Combination for a Strong Thesis Extension

The most impactful and achievable combination for a 6–8 week extension is: **Idea 3** (quick hardware resource numbers — 1 week) followed by **Idea 1** (FPGA prototype — 3 weeks) with **Idea 6** (verification — 1 week) as the correctness argument for the FPGA design. This gives you: a graph showing hardware savings from RigL sparsity, a running FPGA demo you can show at the defence, and a verification section that demonstrates engineering rigour.

Key References to Read First

Reference	Topic	Where to find it
'Digital Design and Computer Architecture' — Harris and Harris	Best beginner textbook for Verilog and FPGA design	Available as PDF from your university library
Evci et al. 'Rigging the Lottery' (2020)	The RigL paper — your baseline	arxiv.org/abs/1911.11134
Nagel et al. 'Data-Free Quantization' (2019)	Post-training quantization — background for FQT	Search on arxiv.org
OpenLane 2 documentation	Full guide to the RTL-to-GDS flow	openlane.readthedocs.io
cocotb documentation	Python hardware testbench framework	docs.cocotb.org
Digilent Arty A7 reference manual	Hardware manual for the recommended FPGA board	digilent.com/reference/programmable-logic/arty-a7